

UNIVERSIDAD NACIONAL DE CÓRDOBA

FACULTAD DE CIENCIAS EXACTAS, FÍSICAS Y NATURALES

Programación Concurrente y Paralela

Granja concurrente de impresión 3D

Grupo: Thread the Needle

Integrantes

Catini, Mariano
Catini, Mateo
Provensale, Lorenzo
Vallenari, Tizziano

1. Recursos compartidos

Todo el estado mutable de la simulación es compartido y vive dentro de un único objeto, el `SimulationMonitor`.

Recurso	Qué contiene	Quién lo modifica
Colas por etapa (`assignmentQueue`, `validationQueue`, `printingQueue`, `qualityControlQueue`)	Las órdenes que esperan en cada etapa	`takeFor...`, `assignPrinter` y `complete...`
Órdenes (`Order`)	Estado, impresora asignada (historial) y contadores por etapa	El monitor, en cada cambio de etapa
Impresoras (`Printer`, en una matriz)	Estado, contador de usos y orden reservada	`assignPrinter` y `complete...`
`processedOrders`	Cantidad de órdenes que llegaron a un estado terminal	`completeValidation` (rechazo), `completePrinting` (falla) y `completeQualityControl`
`cancelled` y `failure`	Si la simulación se canceló y cuál fue la primera causa	`cancel`
`EventLogger`	El archivo `eventos.csv` y el número de secuencia	El monitor, al registrar cada transición

Los workers no comparten datos entre sí: reciben una orden del monitor y le devuelven un resultado. `Order` y `Printer` no tienen lock propio, porque solo se modifican con el lock del monitor tomado. La configuración (`SimulationConfig`) es un registro inmutable y `OutcomeDecider` no guarda estado, osea, solo te devuelve un verdadero o falso de forma pseudoaleatoria así que ninguno de los dos necesita protección.

2. Condiciones de carrera posibles

Sin sincronización habría seis carreras, todas por acceder a la vez al estado compartido.

Carrera	Qué pasaría	Cómo se evita
Dos validadores revisan la cola y toman la misma orden	La orden se validaría dos veces (invariante 3 del enunciado)	`takeFrom` revisa y retira de la cola bajo el mismo lock
Dos asignadores buscan y reservan la misma impresora	Una impresora asignada a dos órdenes (invariante 4)	`assignPrinter` busca y reserva bajo el mismo lock, y `Printer.reserve` exige que esté `AVAILABLE`
`processedOrders++` no es atómico	Se perderían incrementos y la simulación no terminaría	Solo se modifica dentro de métodos `synchronized`

Carrera	Qué pasaría	Cómo se evita
El estado de la orden cambia y la impresora se libera en pasos separados	Una orden terminal podría seguir con la impresora reservada (invariante 6)	El cambio de estado, la liberación, el log y la publicación en la cola siguiente ocurren en un mismo método `synchronized`
Dos hilos escriben en `eventos.csv` a la vez	Filas fuera del orden de confirmación o con secuencia repetida	El log se escribe dentro del lock, que asigna el número de secuencia
Un hilo ve la cola vacía y se duerme justo cuando otro agrega una orden y notifica	El hilo quedaría dormido con trabajo disponible (despertar perdido)	La revisión de la condición y el `wait()` están bajo el mismo lock, y `notifyAll()` se hace también bajo ese lock

3. Estrategia de sincronización

Se usa un único monitor: un solo lock, el de `SimulationMonitor`, protege todo el estado compartido.

Todos los métodos públicos del monitor son `synchronized`, salvo el constructor, que no lo necesita porque los hilos arrancan después de construirlo. Cada worker repite el mismo patrón:

1. **Pedir trabajo, con el lock.** `takeFor...` espera con `wait()` hasta que haya una orden y la retira de su cola.
2. **Trabajar, sin el lock.** El worker aplica la demora con `Thread.sleep` y decide el resultado con `OutcomeDecider`.
3. **Confirmar, con el lock.** `assignPrinter` o `complete...` cambian el estado de la orden y de la impresora, escriben el evento, publican la orden en la cola siguiente y hacen `notifyAll()`.

Solo se usan `Thread`, `Runnable`, `synchronized`, `wait` y `notifyAll`. Las colas son colecciones comunes (`ArrayDeque`, `ArrayList`) que el lock protege, sin clases de `java.util.concurrent` ni `Condition`.

Elegimos un solo lock porque cada operación toca varios recursos a la vez (la cola de origen, la orden, la impresora, el contador, el log y la cola de destino). Con varios locks habría que fijar un orden de adquisición para evitar deadlocks, y con uno solo ese problema no existe. El costo es que las operaciones del monitor se serializan; se compensa dejando las demoras, que son la parte lenta, fuera del lock.

La estructura de clases y la secuencia de cada etapa están en los diagramas de `docs/`.

4. Condiciones de espera y despertar

Un hilo espera cuando no puede avanzar y despierta cada vez que otro hilo cambia el estado compartido.

Dónde espera	Espera mientras	Quién lo despierta
`takeFor...` (los cuatro workers)	Su cola está vacía, `processedOrders < totalOrders` y no hay cancelación	El `notifyAll()` de `assignPrinter`, de los `complete...` o de `cancel`
`assignPrinter`	No hay ninguna impresora `AVAILABLE`	`completeValidation` (rechazo) y `completePrinting` (impresión exitosa), que liberan una impresora, o `cancel`
`awaitCompletion` (hilo principal)	`processedOrders < totalOrders` y no hay cancelación	El `complete...` que lleva `processedOrders` a `totalOrders`, o `cancel`

Si ya no queda ninguna impresora operativa, `assignPrinter` cancela la simulación en lugar de esperar para siempre.

El esquema funciona correctamente gracias a estas 3 reglas:

- **`wait()` siempre dentro de un `while`.** Al despertar se vuelve a revisar la condición: otro worker pudo llevarse la orden, o el `notifyAll()` pudo venir de otra etapa.
- **`notifyAll()` y no `notify()`.** Sin `Condition`, los workers de las cuatro etapas y el hilo principal esperan sobre el mismo objeto. Con `notify()` podría despertar uno que no puede avanzar y dejar dormido al que sí.
- **`wait()` suelta el lock mientras espera** y lo recupera al despertar. Por eso un hilo dormido no impide que otro entre al monitor a dejar trabajo.

5. Mecanismo de terminación

La simulación termina cuando `processedOrders` alcanza `totalOrders`, es decir, cuando todas las órdenes llegaron a un estado terminal, esto incluye órdenes rechazadas o impresiones fallidas.

1. Una orden suma 1 a `processedOrders` al llegar a `REJECTED`, `PRINT_FAILED`, `APPROVED` o `DEFECTIVE`.
2. El `complete...` de la última orden hace `notifyAll()`. Los workers que esperaban en `takeFor...` revisan la condición y reciben `null`.
3. Con `null`, cada worker sale de `run()` y su hilo termina por sí solo.
4. `awaitCompletion` libera al hilo principal, que hace `join` de todos los hilos que creó.
5. Con todos los hilos terminados, `ConcurrentSimulation` cierra el logger, escribe `elementos.csv` y `resumen.properties` y arma el resultado antes de retornar de `execute`.

Los hilos salen siempre por su propia salida de `run()`, sin `Thread.stop` ni `System.exit`.

Cancelación. Si algo falla, por ejemplo la escritura de `eventos.csv`, `cancel(causa)` guarda la primera causa, marca `cancelled` y hace `notifyAll()`. A partir de ahí `takeFor...` devuelve `null`, `assignPrinter` devuelve `false` y `awaitCompletion` retorna, así que ningún hilo queda esperando. Una vez terminados los workers, el monitor puede liberar las reservas pendientes con `releaseReservationsAfterCancellation()`.

6. Análisis teórico del tiempo de ejecución

Con la configuración oficial (500 órdenes, 200 impresoras, 3 / 2 / 3 / 2 hilos por etapa y 85 %, 90 % y 95 % de éxito) se estiman unos 30,31 s, limitados por la validación.

Carga por etapa. Se aproxima con un reparto equilibrado entre los hilos de la etapa:

```
T_{etapa} \approx \frac{\text{órdenes que llegan} \times \text{demora por orden}}{\text{cantidad de hilos}}
```

Las órdenes que llegan salen de los porcentajes: las 500 pasan por asignación y validación, 500 × 0,85 = 425 llegan a impresión y 425 × 0,90 = 382,5 llegan a control de calidad.

Etapa	Órdenes esperadas	Demora (ms)	Hilos	Carga repartida (s)
Asignación	500	50	3	8,33
Validación	500	120	2	30,00
Impresión	425	180	3	25,50
Control de calidad	382,5	80	2	15,30

Tiempo total. Como las etapas se superponen, manda la mayor carga: los 30 s de validación. Sus dos hilos validan unas 16,67 órdenes por segundo (2 / 0,120 s). Las demás etapas tienen capacidad de sobra: impresión procesa hasta 16,67 órdenes por segundo y recibe unas 14,17 (16,67 × 0,85). Sumando los 50 ms de asignación inicial y los 180 ms de impresión y 80 ms de control de calidad de la última orden, el estimado es:

```
T_{estimado} \approx 0{,}050 + 30 + 0{,}180 + 0{,}080 = 30{,}31 \text{ s}
```

Es una aproximación: supone que la última orden validada se imprime y se aprueba, e ignora el costo de las operaciones del monitor.

Estados finales esperados.

Estado final	Cálculo	Esperado
`REJECTED`	500 × 0,15	75
`PRINT_FAILED`	425 × 0,10	≈ 42,5
`DEFECTIVE`	382,5 × 0,05	≈ 19
`APPROVED`	382,5 × 0,95	≈ 363

Las cuatro filas suman 500. Quedarían unas 43 impresoras fuera de servicio y unas 157 operativas de 200, y la suma de los contadores de uso debe dar exactamente 500, porque cada orden se asigna una sola vez. Como `OutcomeDecider` es determinista para una misma semilla, los conteos reales no cambian entre ejecuciones y pueden desviarse algunas órdenes de estos promedios.

7. Resultados de múltiples ejecuciones

Cada ejecución se hace con `java -jar target/tp1.jar` y la configuración oficial, y se anota `durationMillis` y los conteos de `resumen.properties`.

Ejecución	Duración (s)	Aprobadas	Rechazadas	Fallas de impresión	Defectuosas
1	31877 ms	359	71	45	25
2	30494 ms	359	71	45	25
3	30527 ms	359	71	45	25
4	31808 ms	359	71	45	25
5	30559 ms	359	71	45	25

Se espera que los cuatro conteos sean iguales en todas las ejecuciones, porque `OutcomeDecider` es determinista, y que solo cambie la duración.

Ejecución	Duración (s)	Aprobadas	Rechazadas	Fallas de impresión	Defectuosas
6 (Seed de OutcomeDecider modificada)	30486	364	73	43	17

Pendiente: completar la tabla con las corridas reales y agregar el promedio y la desviación de la duración.

8. Comparación entre tiempos teóricos y observados

Se compara el estimado de 30,31 s con el promedio de las ejecuciones de la sección anterior, dentro del rango de 30 a 45 s que fija el enunciado.

Medida	Valor
Tiempo teórico (s)	30,31
Tiempo observado, promedio (s)	31,053
Diferencia (s)	0,743
Diferencia (%)	2,423 %

Se espera que el tiempo observado sea algo mayor que el teórico, por tres motivos:

- `Thread.sleep` duerme al menos el tiempo pedido, y cada hilo lo llama una vez por orden.
- El modelo no cuenta el costo de las operaciones del monitor, que se serializan, ni el de escribir cada evento en `eventos.csv` dentro del lock.
- El modelo supone un reparto perfectamente equilibrado entre los hilos de cada etapa.

Pendiente: completar la tabla con el promedio observado y confirmar cuál de estos motivos explica la diferencia.

9. Efecto de cambiar la cantidad de hilos y las demoras

Agregar hilos solo acelera la etapa que limita: al reducir su carga, el cuello de botella pasa a otra etapa y el tiempo deja de bajar.

Escenario	Asignación (s)	Validación (s)	Impresión (s)	Control de calidad (s)	Cuello de botella	Tiempo estimado (s)
Oficial (3 / 2 / 3 / 2 hilos)	8,33	30,00	25,50	15,30	Validación	30,31
3 validadores	8,33	20,00	25,50	15,30	Impresión	25,75
3 validadores y 4 impresores	8,33	20,00	19,13	15,30	Validación	20,31
1 validador	8,33	60,00	25,50	15,30	Validación	60,31

Cuando manda la validación, el estimado suma 50 ms de asignación inicial y 180 + 80 ms de la última orden. Cuando manda la impresión, suma 50 + 120 ms hasta que llegan las primeras órdenes y 80 ms de control de calidad al final.

- **Hilos de una etapa que no limita.** Subir los hilos de asignación o de control de calidad en la configuración oficial no cambia el tiempo total.
- **Demoras.** El tiempo crece en proporción a la demora de la etapa que limita: con 240 ms de validación y 2 hilos, la carga de validación pasa a 60 s.
- **Demoras nulas.** Sin demoras, el tiempo depende solo del costo del monitor y de escribir el log, que se espera mucho menor que los 30 s.

Pendiente: verificar estas predicciones modificando `config/tp1.properties` y comparando con la duración observada.

10. Justificación de las decisiones de diseño

Cada decisión busca que los invariantes del enunciado se cumplan por construcción y que el diseño sea fácil de explicar.

- **Un solo monitor con un solo lock.** Cada operación toca varios recursos a la vez, y con un solo lock no hay orden de adquisición que cuidar ni deadlock entre locks. El costo es serializar las operaciones del monitor.
- **Entidades pasivas.** `Order` y `Printer` no tienen lock propio: solo se modifican con el lock del monitor tomado, así que otro lock sumaría riesgo sin ganar protección.
- **Una cola por etapa.** Cada worker toma solo de su cola, en orden de llegada, y la etapa anterior la llena. Así las etapas funcionan en paralelo y ninguna espera a que termine la anterior.
- **Demoras y `OutcomeDecider` fuera del lock.** La demora representa el tiempo de trabajo. Dormir con el lock tomado impediría que otros hilos usaran el monitor y se perdería el paralelismo.

- **`wait()` en un `while` y `notifyAll()`.** Despertar no garantiza que la condición se cumpla, y sin `Condition` todas las etapas esperan sobre el mismo objeto.
- **Log dentro del lock.** El número de secuencia y el orden de las filas de `eventos.csv` coinciden con el orden en que se confirman las transiciones, como pide el enunciado. Por eso `EventLogger` no necesita un lock propio.
- **Snapshots inmutables.** El resultado entrega copias de las órdenes y las impresoras tomadas bajo el lock, así que quien las lee no ve estados a medias.
- **Un worker por etapa.** Cada uno usa otros métodos del monitor, otra demora y otra función de `OutcomeDecider`, y asignación tiene un contrato distinto (`assignPrinter` devuelve un `boolean`). Cada clase queda alineada con su diagrama de secuencia, a costa de repetir algo de código.
- **Cancelación con una bandera y `notifyAll()`.** Una falla despierta a todos los hilos para que salgan por su propia salida, sin `Thread.stop` ni `System.exit`.
- **Colecciones comunes.** El enunciado prohíbe las colecciones concurrentes, así que las colas son `ArrayDeque` protegidas por el lock del monitor.